

ASTRIA

AUTONOMOUS COLLISION AVOIDANCE SYSTEM

TECHNICAL WHITEPAPER

Author: Houssam Rharbi - Independent Systems Researcher
Status: Architecture Release - Phase 2 (v2.0)
Domain: Orbital Mechanics, Embedded ML, Network Emulation
Repository: github.com/fDg21G/ASTRIA-Autonomous-Collision-Avoidance

1. ABSTRACT

The proliferation of low-Earth orbit (LEO) constellations necessitates a paradigm shift from ground-in-the-loop collision avoidance to autonomous on-board conjunction screening. ASTRIA is a hardware-in-the-loop (HIL) testbed engineered to evaluate the viability of executing high-fidelity orbital mechanics and machine learning inference directly on edge-compute architectures (ARM Cortex-A).

While Phase 1 of the architecture validated the computational feasibility of sub-millisecond anomaly detection using geometric heuristics, it exposed a critical epistemological flaw: the conflation of 3D Euclidean proximity with collision probability. This whitepaper details the architectural transition to a rigorously statistical framework. By mapping Radial-Transverse-Normal (RTN) covariance to the encounter B-Plane and substituting Euclidean metrics with Mahalanobis distance, the system provides a mathematically sound Probability of Collision (P_c) foundation for hard-real-time embedded operations.

2. SYSTEM ARCHITECTURE (PHASE 1)

The ASTRIA baseline architecture implements an asynchronous, distributed HIL loop modeling the space-to-ground interface. The system decouples state propagation from risk inference to respect edge power constraints.

2.1 Hardware-in-the-Loop Topology

The architecture is bifurcated into two primary nodes communicating via a stateless UDP socket. Transport Control Protocol (TCP) flow-control mechanisms are explicitly bypassed to expose the network layer to raw packet degradation.

Table 1: HIL Node Specifications and Simulated Roles

Node	Hardware	OS / Runtime		Simulated Role
Ground Station	x86-64 Architecture	Ubuntu 22.04	Python 3.11	Mission Control. TLE Ingestion, State Propagation, Uplink.
OBC Emulation	ARM Cortex-A	Native (ARM64)	Linux	Satellite On-Board Computer. Conjunction ML Inference, Downlink.

2.2 Inference Latency & The Geometric Fallacy

The OBC deserializes 68-byte binary payloads containing kinematic vectors and executes an XGBoost gradient-boosted classifier natively on the ARM Floating-Point Unit. Single-pair conjunction inference executes in < 2 ms, and batch screening for 100 pairs completes in < 150 ms.

However, Phase 1 utilizes Euclidean distance ($D = \|\vec{r}_1 - \vec{r}_2\|_2$) in $\mathbb{R}^3$ Cartesian space as the primary metric. This conflates geometric proximity with statistical risk. A 100 m miss distance between precisely tracked objects yields near-zero risk, whereas a 5 km miss distance involving high-uncertainty state vectors can cross the critical operational threshold. Fast spatial queries (e.g., KD-Trees) in Euclidean space optimize proximity search in a fundamentally incorrect risk space.

3. THE PROBABILISTIC FRAMEWORK (PHASE 2)

To achieve operational validity, ASTRIA transitions to stochastic mechanics. Phase 2 introduces error ellipsoid modeling, planar projection, and probability-based classification.

3.1 Empirical RTN Covariance Assignment

Standard Two-Line Elements (TLEs) lack embedded covariance data. The architecture injects synthetic error models based on empirical radar tracking residuals, parameterized in the Radial-Transverse-Normal (RTN) frame.

$$C_{RTN} = \text{diag}(\sigma_R^2, \sigma_T^2, \sigma_N^2) \quad (1)$$

3.2 B-Plane Projection and Probability of Collision

The combined positional covariance ($C_{comb} = C_{RTN,1} + C_{RTN,2}$) is linearly projected onto the B-Plane—the plane orthogonal to the relative velocity vector ($\vec{v}_{rel}$) at the Time of Closest Approach (TCA), defined by basis vectors ξ and η . This yields a 2×2 covariance matrix C_{xy} .

The Probability of Collision (P_c) is computed by integrating the bivariate normal density function f over the hard-body cross-section S of the two objects:

$$P_c = \iint_S f(x, y; C_{xy}) dx dy \quad (2)$$

Table 2: Empirical Uncertainty Parameters for LEO State Vectors (TLE Age ≤ 24 h)

Parameter	Symbol	Nominal Value	Primary Physical Driver
Radial uncertainty (1σ)	σ_R	~ 100 m	Solar Radiation Pressure model error
Transverse uncertainty (1σ)	σ_T	~ 500 m	Along-track timing drift (SGP4 drag)
Normal uncertainty (1σ)	σ_N	~ 200 m	Inclination determination error
Cross-correlation (R-T)	ρ_{RT}	< 0.3	Observational arc length
TLE age decay factor	$k(\Delta t)$	$\exp(\Delta t/\tau)$	Atmospheric drag and maneuver cadence

where S is a disk defined by the sum of the objects' radii ($r = r_1 + r_2$). The operational target threshold is $P_c \geq 1 \times 10^{-4}$.

3.3 Mahalanobis Distance and Feature Engineering

The core machine learning feature is upgraded from Euclidean distance to Mahalanobis distance (D_M) within the B-Plane. Given a miss vector $\vec{b} = (b_\xi, b_\eta)$, D_M scales the spatial offset by the combined uncertainty tensor:

$$D_M = \sqrt{\vec{b}^T C_{xy}^{-1} \vec{b}} \quad (3)$$

Table 3: Evolution of Machine Learning Feature Space

Feature	Description	Phase 1	Phase 2
Euclidean Miss Distance	Distance in ECI ($\ \vec{r}_1 - \vec{r}_2\ _2$)	Primary	Diagnostic
Relative Velocity	Magnitude of $\vec{v}_{rel}$ in km/s	Active	Active
Mahalanobis Distance (D_M)	Dimensionless risk metric ($\sqrt{\vec{b}^T C_{xy}^{-1} \vec{b}}$)	–	Primary
$\sigma_T / \ \vec{b}\ $ Ratio	Elongation of uncertainty vs. miss vector	–	Active
TLE Age	Proxy scalar for covariance growth	–	Active
Hard-Body Radius Sum	Target cross-section threshold ($r_1 + r_2$)	–	Active
B-Plane Angle	Geometric orientation of the miss vector	–	Active

4. CHANNEL EMULATION & HIL REALISM

A functional local-area network layer is structurally insufficient for space systems validation. Phase 2 introduces a stateless channel emulation proxy that injects deterministic, physics-based RF degradation into the UDP link.

Table 4: RF Link Impairment Emulation Models

Impairment	Stochastic / Physical Model	Key Architectural Parameters
Packet Loss (Bursty)	Gilbert-Elliott 2-state Markov chain	$\alpha = 0.05, \beta = 0.20, P_B = 0.30$
Propagation Delay	Geometric slant-range kinematics	Altitude $h = 550$ km, Elev. mask 10°
Delay Jitter	Gaussian additive noise on $\tau(t)$	$\sigma_{jitter} = 0.3$ ms
Packet Reordering	Delay inversion delivery matrices	$P_{reorder} = 0.01$

4.1 Kinematic Delay and Doppler Shift

One-way signal latency is dynamically calculated based on instantaneous slant range. For a 550 km LEO orbit, delay oscillates between 1.8 ms (zenith) and 7.3 ms (10° elevation). Slant range $\tau(t)$ is derived geometrically:

$$\tau(t) = \frac{\sqrt{R_E^2 + (R_E + h)^2 - 2R_E(R_E + h)\cos\theta(t)}}{c} \quad (4)$$

The emulation layer models corresponding Doppler frequency shift proportional to the line-of-sight velocity vector (v_{los}), setting the foundation for future SDR integration at S-band reference frequencies (f_c):

$$\Delta f = f_c \cdot \frac{v_{los}(t)}{c} \quad (5)$$

5. DEPLOYMENT ROADMAP

To transition from analytical prototyping to a flight-ready paradigm, the execution environment requires strict temporal determinism. The high-level Python stack will be deprecated in favor of native architectures.

6. ARCHITECTURAL BOUNDARY CONDITIONS

The following constraints define the operational perimeter of the current system and dictate the trajectory of future engineering iterations:

- **Radiation Tolerance Deficit:** Emulation is executed on consumer ARM silicon lacking SRAM error-correcting codes (ECC), triple modular redundancy (TMR), or CMOS latch-up immunity. Total Ionizing Dose (TID) degradation is fundamentally unmodeled.
- **Execution Parallelization:** The current inference loop is strictly sequential. Scaling to evaluate a full LEO catalog ($> 20,000$ tracked objects) requires implementing hierarchical broad-phase filtering and multithreaded tensor execution.

Table 5: System Migration Milestones

Milestone	Description	Dependency
M1: P_c Integrator	Numerical P_c evaluation via Patera series expansion	Phase 2 Covariance
M2: Dataset Regen.	Retrain XGBoost classifier on P_c -labeled synthetic data	M1
M3: Channel Emulator	Integration of Gilbert-Elliott, delay, and Doppler models	Phase 1 UDP
M4: ONNX Export	Serialize XGBoost graph via sklearn-onnx	M2
M5: Native UDP Layer	ARM64 cross-compiled C++ payload deserialization	M4
M6: ONNX RT Integ.	C++ inference via ONNX Runtime natively on ARM	M4, M5
M7: Bare-Metal Rust	Full OBC stack migrated to Rust (<code>#[no_std]</code>)	M6 Validation

- **Control-Theoretic Void:** The system isolates risk classification from response. Optimization of impulsive avoidance maneuvers under rigid fuel (ΔV) and attitude pointing constraints is excluded from the current pipeline.
- **Dynamical Uncertainty Limitations:** Atmospheric drag variance (B^* term) introduces massive along-track uncertainty for sub-400 km geometries. Furthermore, the architecture relies on static empirical covariance assignment rather than rigorous State Transition Matrix (STM) forward propagation.

7. REFERENCES

1. Hoots, F. R., & Roehrich, R. L. (1980). Models for Propagation of NORAD Element Sets. Space-track Report No. 3. Project SPACETRACK, USAF.
2. Vallado, D. A., Crawford, P., Hujsak, R., & Kelso, T. S. (2006). Revisiting Spacetrack Report #3: Rev 2. AIAA 2006-6753.
3. Foster, J. L., & Estes, H. S. (1992). A Parametric Analysis of Orbital Debris Collision Probability and Maneuver Rate for Space Vehicles. NASA/JSC-25898.
4. Alfano, S. (2005). A Numerical Implementation of Spherical Object Collision Probability. Journal of Astronautical Sciences, 53(1), 103-109.
5. Patera, R. P. (2001). General Method for Calculating Satellite Collision Probability. Journal of Guidance, Control, and Dynamics, 24(4), 716-722.
6. Flohrer, T., Krag, H., & Klinkrad, H. (2008). Assessment and Categorization of TLE Orbit Errors for the US SSN Catalogue. ESA SP-672.
7. Vallado, D. A., & Cefola, P. J. (2012). Two-line Element Sets Practice and Use. 63rd International Astronautical Congress, IAC-12-B4.2.5.
8. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD '16, ACM.
9. Elliott, E. O. (1963). Estimates of Error Rates for Codes on Burst-Noise Channels. Bell System Technical Journal, 42(5), 1977-1997.

10. ONNX Runtime Development Team. (2019). ONNX Runtime: Cross-Platform, High Performance ML Inferencing. <https://onnxruntime.ai>
11. Kelso, T. S. Celestrak SATCAT and TLE Archive. <https://celestrak.org>
12. Space-Track.org. 18th Space Control Squadron, USAF. <https://www.space-track.org>